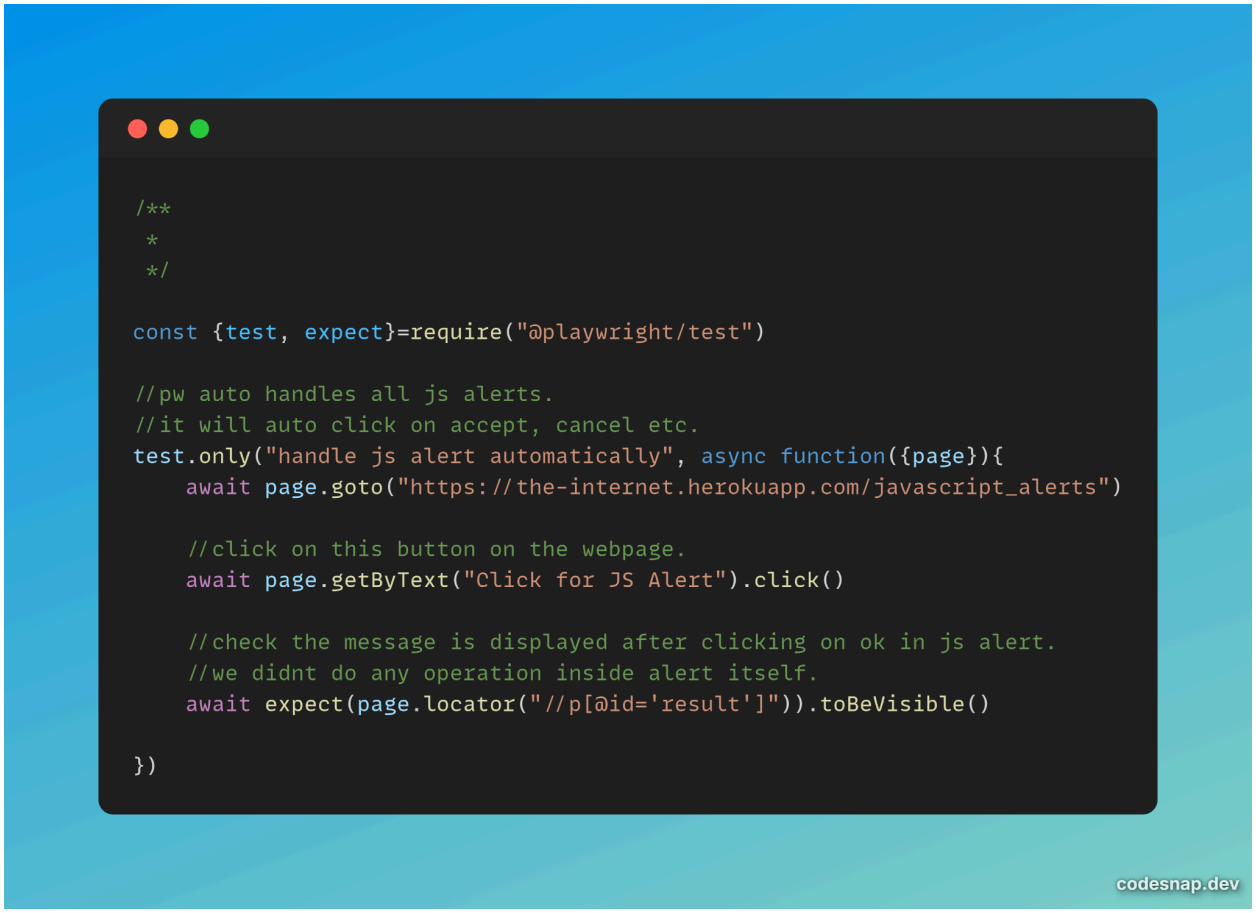
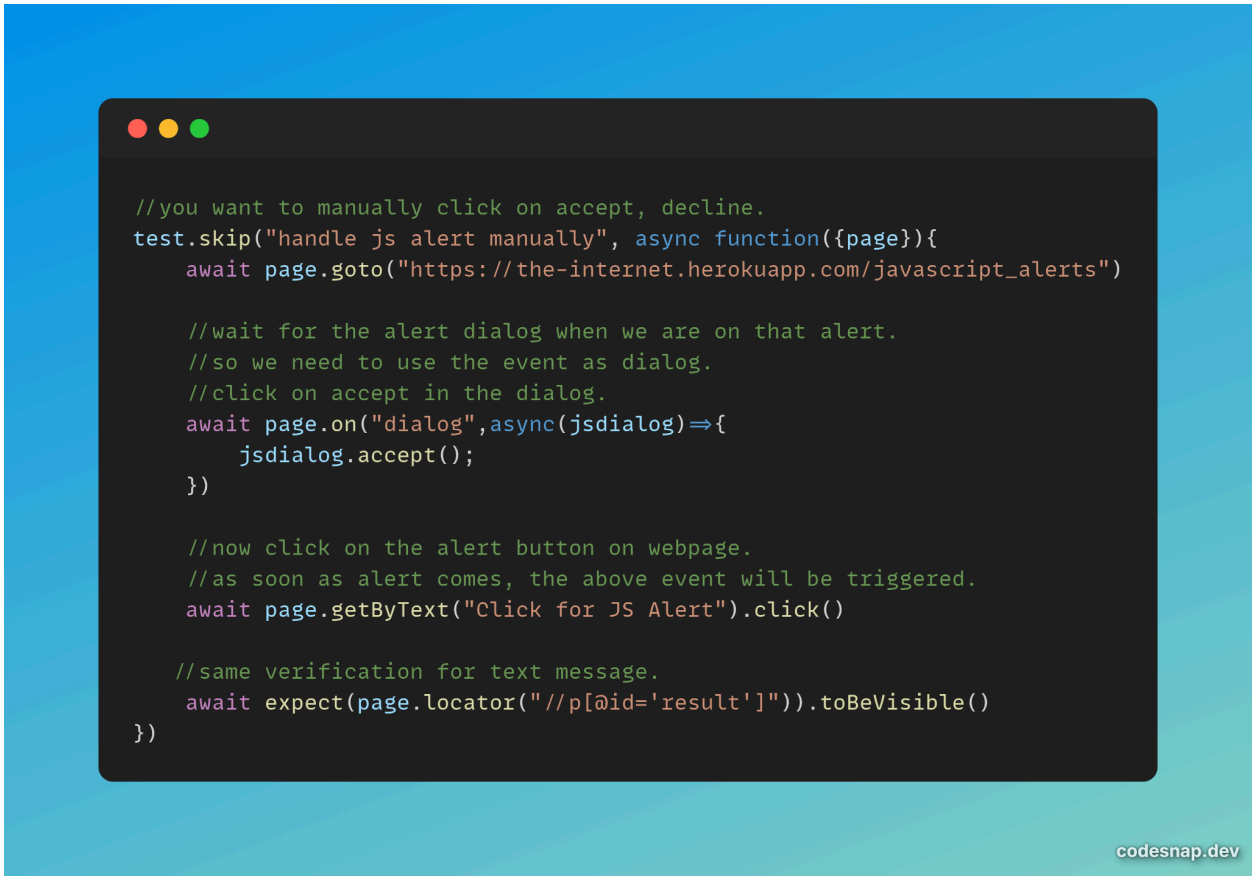
pw-19th nov-4

—js alerts

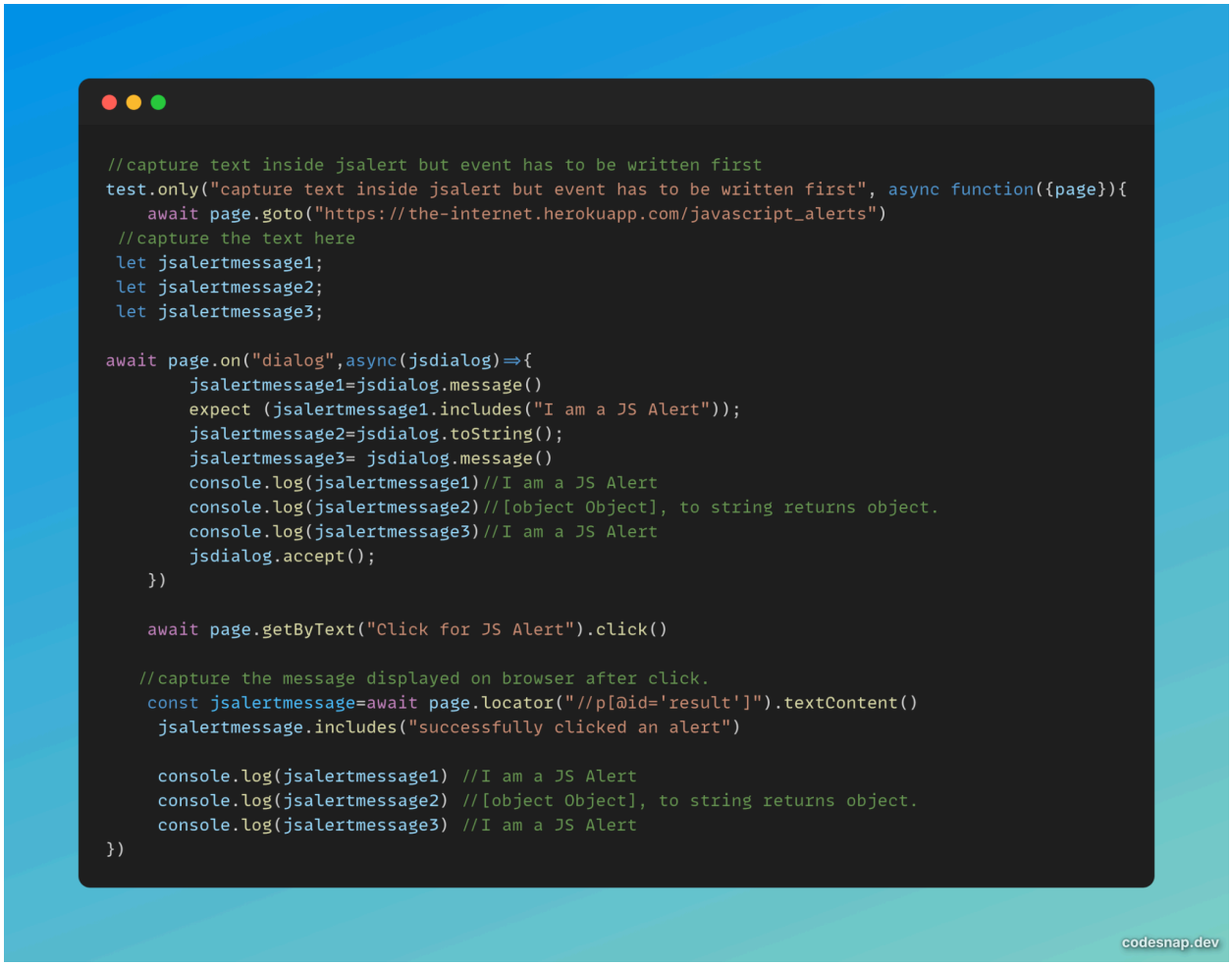


```
/**
 *
 */

const {test, expect}=require("@playwright/test")

//pw auto handles all js alerts.
//it will auto click on accept, cancel etc.
test.only("handle js alert automatically", async function({page}){
    await page.goto("https://the-internet.herokuapp.com/javascript_alerts")

    //click on this button on the webpage.
    await page.getByText("Click for JS Alert").click()

    //check the message is displayed after clicking on ok in js alert.
    //we didnt do any operation inside alert itself.
    await expect(page.locator("//p[@id='result']")).toBeVisible()

})
```


//you want to manually click on accept, decline.
test.skip("handle js alert manually", async function({page}){
await page.goto("https://the-internet.herokuapp.com/javascript_alerts")
//wait for the alert dialog when we are on that alert.
//so we need to use the event as dialog.
//click on accept in the dialog.
await page.on("dialog",async(jsdialog)=>{
jsdialog.accept();
})
//now click on the alert button on webpage.
//as soon as alert comes, the above event will be triggered.
await page.getByText("Click for JS Alert").click()
//same verification for text message.
await expect(page.locator("//p[@id='result']")).toBeVisible()
})
codesnap.dev


//capture text after clicking on js alert
//location of the event can be before clicking on alert only, else when we try to capture the values from alert we get undefined.
test.only("capture text after clicking on js alert", async function({page}){
await page.goto("https://the-internet.herokuapp.com/javascript_alerts")
await page.getByText("Click for JS Alert").click()
//location works even after when we write after clicking on button.
await page.on("dialog",async(jsdialog)=>{
jsdialog.accept();
})
//capture the message displayed on browser after click.
const jsalertmessage=await page.locator("//p[@id='result']").textContent()
jsalertmessage.includes("successfully clicked an alert")
})
codesnap.dev

```
//capture text inside the js alert
test.only("capture text inside the js alert", async function({page}){
    await page.goto("https://the-internet.herokuapp.com/javascript_alerts")


    await page.getByText("Click for JS Alert").click()

 //capture the text here
 let jsalertmessage1;
 let jsalertmessage2;
 let jsalertmessage3;
    await page.on("dialog",async(jsdialog)=>{
        jsalertmessage1=jsdialog.message()
        expect (jsalertmessage1.includes("I am a JS Alert"));
         jsalertmessage2=jsdialog.toString();
        jsalertmessage3= jsdialog.message()
        console.log(jsalertmessage1)
        console.log(jsalertmessage2)
        console.log(jsalertmessage3)
        jsdialog.accept();
    })

   //capture the message displayed on browser after click.
    const jsalertmessage=await page.locator("//p[@id='result']").textContent()
     jsalertmessage.includes("successfully clicked an alert")

     console.log(jsalertmessage1) //undefined, need to handle dialog before event so it captures value.
     console.log(jsalertmessage2) //undefined
     console.log(jsalertmessage3) //undefined
})
```



```
//capture text inside jsalert but event has to be written first
test.only("capture text inside jsalert but event has to be written first", async function({page}){
    await page.goto("https://the-internet.herokuapp.com/javascript_alerts")
 //capture the text here
 let jsalertmessage1;
 let jsalertmessage2;
 let jsalertmessage3;

await page.on("dialog",async(jsdialog)⇒{
        jsalertmessage1=jsdialog.message()
        expect (jsalertmessage1.includes("I am a JS Alert"));
        jsalertmessage2=jsdialog.toString();
        jsalertmessage3= jsdialog.message()
        console.log(jsalertmessage1)//I am a JS Alert
        console.log(jsalertmessage2)//[object Object], to string returns object.
        console.log(jsalertmessage3)//I am a JS Alert
        jsdialog.accept();
    })

    await page.getByText("Click for JS Alert").click()

   //capture the message displayed on browser after click.
    const jsalertmessage=await page.locator("//p[@id='result']").textContent()
     jsalertmessage.includes("successfully clicked an alert")

     console.log(jsalertmessage1) //I am a JS Alert
     console.log(jsalertmessage2) //[object Object], to string returns object.
     console.log(jsalertmessage3) //I am a JS Alert
})
```

only line change for clicking cancel in js confirm —

listen1.dismiss()

—

in js confirm and js prompt when we dont use event handlers then js clicks on cancel button.

—one line change for clicking on cancel in js prompt-

listen1.dismiss()

—To limit workers when running- go through official documentation

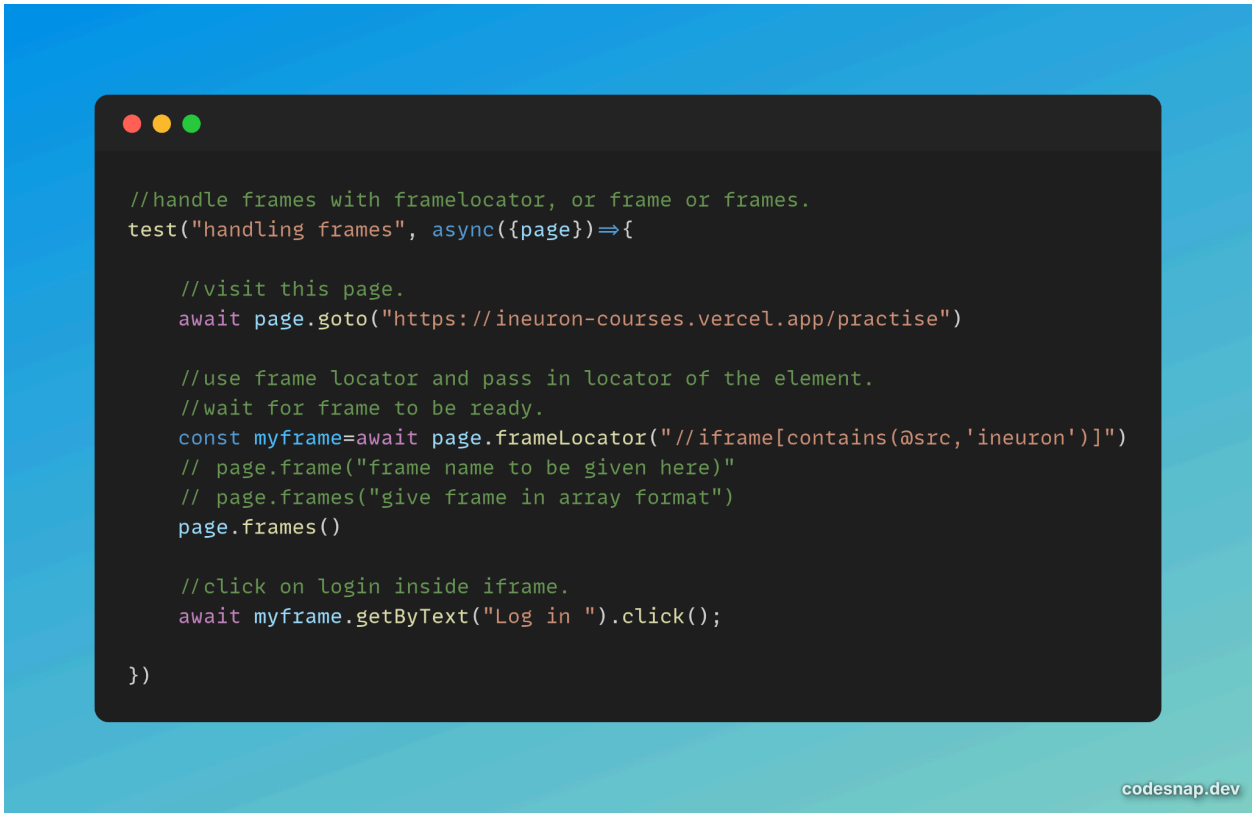
Disable more than one worker-

```
export default config;
```

Disable parallelism

You can disable any parallelism by allowing just a single worker at any time. Either set `workers: 1` option in the configuration file or pass `--workers=1` to the command line.

```
npx playwright test --workers=1
```



—handling nested frames

only area to be changed.

```
   //wait for first frame.
     let f1=await page.frameLocator("//iframe[contains(@src,'ineuron')]")
 var f2= f1.frameLocator('second frame')
 const f3= f2.frameLocator('third frame')

 //then perform operation on the frames and to come out of frames
 //no need of switch
 //directly use page because it points to the original page.
```

codesnap.dev

```
//wait for load state with network idle, load, dom load options.
test("wait for all checkbox apis to load on the webpage and then proceed.", async({page})=>{

    await page.goto("https://ineuron-courses.vercel.app/login");

    await page.waitForTimeout(5000)

await page.getByText("New user? Signup").click()

//there are three options.
//dom to be loaded.
//network idle till apis are loaded.
//load method till page loads.
await page.waitForLoadState("networkidle")

//we have to use wait for selector to load all the apis on the page else error.
await page.waitForSelector("//label[@class='interest']")

//we will capture all interest in a locator and get the count.
let interestlocator=await page.locator("//label[@class='interest']").count()
console.log(interestlocator)
await page.waitForTimeout(5000)
expect (interestlocator).toBe(12);
})
```

codesnap.dev